

scripts/pipeline/phase-02-test-go.sh — User Guide

Last verified: 2026-08-25 (run-isolation seam LAVA_STRESS_CHAOS_EVIDENCE_DIR, added 2026-08-23) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate), §11.4.85 (stress + chaos tests); Lava §6.A (real-binary contract tests), §6.J (Anti-Bluff), §6.T.2 (resource limits)

Overview

The build-test-distribute pipeline's **Go test wrapper**. It runs the lava-api-go module's own test suite once and turns the result into per-test Evidence Records, covering two of the eight FR-002 categories:

Category	Which tests
real-binary-contract	every test whose <code>go test -json</code> Package is, or is nested under, <code>digital.vasic.lava.apigo/tests/contract</code> — the §6.A tests that exercise real built binaries (<code>cmd/lava-api-go</code> , <code>cmd/healthprobe</code>) and real <code>compose/Dockerfile</code> fixtures
go-unit-integration	every other Go test in the module

Category membership is decided **per test, from the real Package field Go itself reports**, never from a static list.

There is one `go test` invocation, not two. `lava-api-go/tests/contract/` has no `go.mod` of its own — it is an ordinary package of the single `digital.vasic.lava.apigo` module — so the project's own `Makefile` `test: target` (`GOMAXPROCS=2 go test -race -count=1 ./...`) already runs it. This wrapper reuses those exact flags and appends `-json`, which changes only Go's output format, not which tests run or with what result.

Usage

scripts/pipeline/phase-02-test-go.sh [repo-path] [phase-dir]

Positional	Required	Meaning
repo-path	no	Lava monorepo root. Defaults to <code>git rev-parse --show-toplevel</code> .
phase-dir	no	The phase-02 evidence directory this run writes under. Defaults to a freshly generated <code><repo-path>/.lava-ci-evidence/pipeline-runs/<UTC-run-id>/phase-02</code> , so the script is independently runnable per FR-005 even without the orchestrator passing a shared run's phase dir.

Outputs

Everything is written under `<phase-dir>/`:

Path	Contents
<code><phase-dir>/<category>/<test_id>.json</code>	one Evidence Record per individual Go test, including each named subtest
<code><phase-dir>/raw/go/</code>	that one test's own real captured stdout, per test
<code><phase-dir>/raw/go-test-json-stream.jsonl</code>	the raw <code>go test -json</code> event stream
<code><phase-dir>/raw/go-test-stderr.log</code>	the run's stderr
<code><phase-dir>/raw/go-stress-chaos-evidence/</code>	the suite's own §11.4.85 stress/chaos evidence — see Run isolation below

`assertion_summary` is never a generic phrase: a PASS quotes the real --- PASS: <name> (<elapsed>s) line Go printed; a FAIL quotes the real <file>.go:<line>: <message> assertion text.

The script writes only Evidence Records and their raw companions. **It never touches git state, never pushes, and never distributes anything.**

Run isolation — a run must not dirty the working tree

This is the behaviour added 2026-08-23, and it is why the wrapper sets an environment variable before invoking `go test`.

The defect

The first genuine end-to-end pipeline run proved the go category **could only ever run once**.

Two of lava-api-go's own \$11.4.85 stress/chaos test files resolve their evidence directory by walking **up** from the test's working directory until they find a `.lava-ci-evidence` directory, then writing into its `stress-chaos/jackett/` subdirectory:

- `lava-api-go/internal/jackett/stress_chaos_test.go` → `evidenceDir()`
- `lava-api-go/internal/handlers/v1/jackett_stress_chaos_test.go` → `handlerEvidenceDir()`

Six files land there, and they are **tracked and not gitignored** — `git ls-files --error-unmatch` succeeds on each, `git check-ignore` exits 1:

```
.lava-ci-evidence/stress-chaos/jackett/chaos-categorized-handler-jackett.json
.lava-ci-evidence/stress-chaos/jackett/chaos-categorized-jackett.json
.lava-ci-evidence/stress-chaos/jackett/stress-chaos-search-thundering-herd.json
.lava-ci-evidence/stress-chaos/jackett/stress-chaos-search-upstream-faults.json
.lava-ci-evidence/stress-chaos/jackett/stress-handler-concurrent-jackett.json
.lava-ci-evidence/stress-chaos/jackett/stress-latency-jackett.json
```

Their payload carries `captured_at` plus measured latency percentiles, so the content differs on **every** run by construction — this is not an occasional collision, it is guaranteed.

Net effect: a **successful** run left the working tree dirty, and the next invocation was refused by the pipeline's FR-000 precondition with exit 2, working tree is not clean. Observed verbatim: run 2026-08-23T10-17-26Z succeeded, run 2026-08-23T10-31-21Z was refused. That also falsified `quickstart.md` Scenario 5's claim that

everything a run produces is gitignored, and it blocked FR-018 (every run restarts from scratch) and SC-007. A pipeline that can only ever run once is not a pipeline.

The fix

Both resolvers now honour a LAVA_STRESS_CHAOS_EVIDENCE_DIR environment seam. When it is set to a non-empty value they MkdirAll it and write there; when it is unset they fall back to the historical walk-up, **byte for byte**. A plain go test or make test is therefore completely unaffected.

This wrapper sets it to <phase-dir>/raw/go-stress-chaos-evidence/:

```
STRESS_CHAOS_EVIDENCE_DIR="${PHASE_DIR}/raw/go-stress-chaos-  
evidence"  
mkdir -p "$STRESS_CHAOS_EVIDENCE_DIR"  
...  
export LAVA_STRESS_CHAOS_EVIDENCE_DIR="$STRESS_CHAOS_EVIDENCE_DIR"
```

The default phase dir lives under .lava-ci-evidence/pipeline-runs/, which **is** gitignored (.gitignore:59). The evidence is still produced in full — it simply lands with the rest of the run's artifacts instead of overwriting a tracked file, which is how every other test category already behaved.

Note the shape of the fix: the evidence is **relocated, never suppressed**. Deleting the writes would have made the working tree clean too, and would have destroyed the \$11.4.85 evidence while doing so.

Regression coverage

tests/pipeline/test_phase_02_go_evidence_isolation.sh — hermetic, with no real Go toolchain and no real lava-api-go. A stub go on PATH replays a captured real go test -json stream **and** reproduces the side effect, by transcribing the real Go resolver into shell line for line: walk up for .lava-ci-evidence, else fall back. The stub is never told the answer, so if the wrapper does nothing the stub resolves to the tracked path exactly as the real tests do.

Case	Asserts
1 (load-bearing)	a go-category run modifies no tracked file in the working tree
2 (positive / anti-vacuous)	the stress/chaos evidence is still written , inside the run's own directory. A blanket "write nothing ever"

Case	Asserts
	change would satisfy case 1 while destroying the evidence; case 2 rejects it
3 (end-to-end)	FR-000 still passes after a run, so a second run is accepted — the property the defect actually broke

Honest non-execution: skips are reported, never force-PASSed

lava-api-go has no Gradle-style `-Pintegration=true` flag. Its equivalent is the `POSTGRES_TEST_URL` environment variable; tests needing a real Postgres (TestPostgresStorageConformance, TestCrossBackendParity, TestNilEmptyContract_Postgres) call `t.Skip(...)` when it is unset.

This wrapper deliberately does **not** set `POSTGRES_TEST_URL` or invoke `scripts/run-test-pg.sh`. At authoring time this host already ran a different, long-lived lava-postgres container that is the project's **live application database**, not a disposable fixture. Repurposing a live application database as a test target is a production dependency this script has no business touching.

So `POSTGRES_TEST_URL` stays unset, the gated tests self-report SKIP, and each one gets its own SKIPPED Evidence Record quoting Go's own captured skip reason verbatim — anti-bluff-validated exactly like any PASS or FAIL record, and enumerated separately in the final summary (SKIP (n): ...). Writing a PASS for a test that never executed would be precisely the Sixth/Seventh Law violation this project forbids.

Failure modes that are recorded rather than absorbed

- **A package-level build/compile failure or a fatal panic** that kills a test binary before any individual test reports an outcome gets its own synthetic Evidence Record (`test_id` suffixed `#{build}`) carrying the compiler's or runtime's own diagnostic, instead of vanishing into "0 tests recorded".
- **A non-zero go test exit that no Evidence Record explains** is itself a failure (added 2026-08-21). The canonical shape is `TestMain` teardown after `m.Run()` returning 0 — every test PASSes and the package still fails — which had been recorded as all-PASS while go

test genuinely exited non-zero. Now a synthetic FAIL record quotes the captured package-level output.

Exit codes

Code	Meaning
0	go test ran, at least one per-test outcome was parsed, every Evidence Record was written and validated, every recorded result was PASS, and go test's own exit code was 0 (or non-zero and fully explained by a recorded FAIL). Real SKIPS do not affect this.
1	At least one Go test genuinely reported FAIL, at least one Evidence Record was REJECTED by anti-bluff validation, or go test exited non-zero with no recorded FAIL explaining it.
2	Usage/precondition error — repo path, lava-api-go/ directory, or lava-api-go/Makefile missing.
3	Internal wrapper error — a required tool (jq, python3, go) is missing, or go test -json produced zero parseable per-test events (the Go toolchain failed before running any test, or no test was compiled into the run at all; a stray build tag excluding every _test.go is the realistic trigger). The real stderr is printed for diagnosis.

Maintenance

scripts/pipeline/** is **outside** the CM-SCRIPT-DOCS-SYNC gate's matching set — scripts/check-script-docs-sync.sh pairs find scripts -maxdepth 1 -name '*.sh' against find docs/scripts -maxdepth 1 -name '*.sh.md', so neither this script nor this document is counted by it. The §11.4.18 obligation to keep them in sync still applies; it is simply not mechanically enforced here. Update this document in the same commit that changes the script.

Every .md under docs/ is in scope for CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC (§11.4.65), so regenerate the siblings after editing:

```
bash scripts/sync-markdown-exports.sh --regenerate docs/scripts/  
    pipeline/phase-02-test-go.sh.md  
bash scripts/check-markdown-export-sync.sh
```

Cross-references

- `scripts/pipeline/phase-02-test-go.sh` — the script itself
- `tests/pipeline/test_phase_02_go_evidence_isolation.sh` — the run-isolation suite
- `tests/pipeline/test_phase_02_go_wrapper.sh` — the wrapper's parsing/exit-code suite (CASE 2, CASE 3)
- `lava-api-go/internal/jackett/stress_chaos_test.go` — `evidenceDir()`, one of the two seam sites
- `lava-api-go/internal/handlers/v1/jackett_stress_chaos_test.go` — `handlerEvidenceDir()`, the other
- `scripts/pipeline/lib/evidence.sh` — `write_evidence_record`
- `scripts/pipeline/lib/anti-bluff-validate.sh` — `validate_evidence_record`
- `specs/002-build-test-distribute-pipeline/contracts/evidence-record.schema.json` — the record schema
- `specs/002-build-test-distribute-pipeline/data-model.md` — the Evidence Record entity